A Multi-Lane Pure Pursuit Controller for Head-to-Head F1TENTH Autonomous Racing

Kaifan Yu
School of Engineering and Applied Science
University of Pennsylvania
Philadelphia, PA, USA
kaifany@seas.upenn.edu

***Abstract*—This paper presents the design, implementation, and evaluation of a multi-lane pure pursuit controller for a 1/10-scale F1TENTH autonomous racing vehicle competing in head-to-head time trials and races. The pipeline consists of (i) offline mapping with the ROS 2 `slam_toolbox` package driven by manual teleoperation at 1 m/s, (ii) high-rate Monte Carlo localization through a forked particle filter, and (iii) two manually curated racelines authored in a custom Python + Matplotlib waypoint editor that overlays the SLAM map and allows per-waypoint velocity assignment. At runtime, an adaptive-lookahead pure pursuit controller is augmented with three additions: a LiDAR-based two-frame opponent confirmation tracker that clusters consecutive scan returns into vehicle-sized objects and rejects transient detections; a cost-based lane-switching policy that balances a smooth collision-clearance term, a leftward racing-line preference, and a switching penalty under a cooldown timer; and a forward-window speed-scaling rule that decelerates gracefully when an opponent occupies the active lane. The complete stack runs onboard an NVIDIA Jetson Orin Nano Super with a 2D Sick LiDAR and a Traxxas chassis driven by a VESC speed controller, and was validated both in the F1TENTH simulator and on hardware. The vehicle achieved a best lap time of 21 s during a competitive race on the course. We document the design choices, parameter selection, hand-tuned velocity profile, and observed failure modes — most notably oscillatory lane switching when no opponent is present and a side-on collision blind spot inherent to a planar LiDAR — and outline a path toward learning-based opponent prediction.**

***Index Terms*—F1TENTH, autonomous racing, pure pursuit, multi-lane planning, LiDAR clustering, particle filter, ROS 2.**

I. Introduction

The F1TENTH autonomous racing platform provides a fast, accessible testbed for research and education in real-time motion planning and control. A successful head-to-head racing agent must, at minimum, (a) localize precisely on a pre-mapped track at high update rates, (b) follow a near-time-optimal racing line, and (c) detect a moving opponent quickly enough to avoid collision while still making progress around the lap. Two mature high-level architectures dominate the literature: model predictive control (MPC) tracking a reference trajectory subject to dynamic and workspace constraints, and geometric pure pursuit acting on a curated set of waypoints. MPC offers a principled way to respect dynamic limits but pays for it with non-trivial QP solver overhead and noticeable sensitivity to constraint construction; pure pursuit [1] is comparatively trivial to implement, robust, and amenable to high-rate operation on embedded compute, but lacks any built-in mechanism for obstacle avoidance.

This work pursues the pure pursuit branch of that design space and extends it along a single dimension: instead of tracking one fixed waypoint sequence, the controller selects on every iteration between several pre-authored *lanes* that span the same lap. Lanes are aligned by index so that lane ℓ at index i is the spatial sibling of lane ℓ' at the same index; this turns lane selection into a discrete, three-way (or two-way) cost minimization that is evaluated at every pose update. A LiDAR clustering stage proposes opponents at each scan, a two-frame persistence filter rejects transient detections, and the resulting opponent positions are projected onto each lane's forward window to score collision risk.

The remainder of the paper is organized as follows. Section II describes the hardware platform and software stack. Section III covers offline mapping. Section IV briefly describes the localization pipeline. Section V details the waypoint generation and curation workflow, including the interactive editor and the velocity assignment heuristic that gave the largest single improvement in lap time. Section VI is the technical core: it defines the multi-lane pure pursuit controller with full equations and pseudocode. Section VII shows how the pieces are wired together as ROS 2 nodes. Section VIII presents results, Section IX catalogs the failures we did and did not solve, and Section X sketches future work.

II. Hardware and Software Setup

A. Vehicle platform

The vehicle is a stock F1TENTH-class build on a Traxxas 1/10-scale chassis with a brushed DC drive motor and a steering servo. Throttle and steering signals are dispatched by a VESC electronic speed controller, which receives ROS `AckermannDriveStamped` commands through the standard `vesc_ackermann` bridge. Onboard compute is an NVIDIA Jetson Orin Nano Super (8 GB RAM, 256 GB NVMe SSD) running Ubuntu 22.04 and ROS 2 Humble. Perception is provided by a single 2D Sick LiDAR mounted at the front. Manual teleoperation, used during mapping and ad-hoc testing, is provided by a Sony DualShock 4 controller. The chassis can reach approximately 40 mph (17.9 m/s) at full throttle; the controller in this work is intentionally limited to a small fraction of that envelope. Table I summarizes the platform.

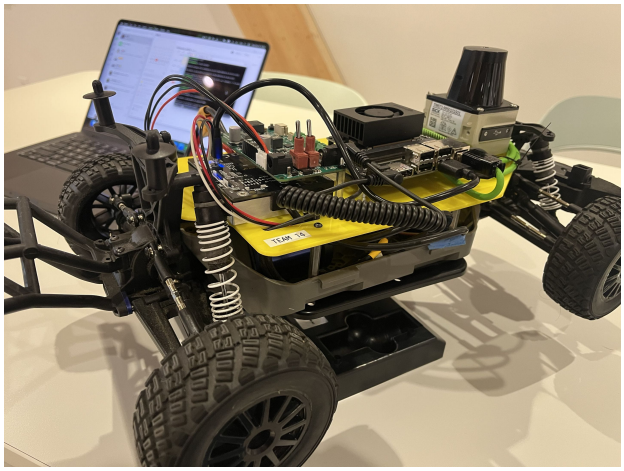

Fig. 1: The F1TENTH vehicle used in this work.

TABLE I: Hardware Platform Summary

Component	Specification
Chassis	Traxxas 1/10-scale, brushed DC drive
Steering	Servo, $\delta_{\max} \approx 24°$ (0.4189 rad)
Wheelbase	0.3302 m
Speed controller	VESC (open-loop → Ackermann bridge)
LiDAR	2D Sick (planar, front-mounted)
Compute	NVIDIA Jetson Orin Nano Super 8 GB LPDDR5, 256 GB NVMe SSD
Operating system	Ubuntu 22.04 LTS
Middleware	ROS 2 Humble
Teleop	Sony DualShock 4
Top mechanical speed	≈ 40 mph (17.9 m/s)

B. Software stack

All nodes are written in C++ (`rclcpp`) with the exception of the waypoint editor, which is a Python + Matplotlib tool used offline. The runtime stack uses three external packages from source: `slam_toolbox` for mapping, a fork of the MIT-RACECAR `particle_filter` package for localization [1][1], and the standard `ackermann_msgs` drive interface. The simulator used during early development is the F1TENTH ROS 2 gym, which exposes the same topics as the hardware bring-up (`/scan`, `/ego_racecar/odom`, `/drive`) so that controllers transfer between the two with no code change beyond the `use_odom` parameter.

III. Mapping with SLAM Toolbox

A static occupancy grid map of the track is produced offline using `slam_toolbox` in `online_async` mode. The vehicle is driven manually with the DualShock 4 at a deliberately conservative top speed of **1 m/s**. We empirically observed that higher teleoperation speeds introduced visible scan registration drift on the long, low-feature straights of the track and produced maps in which the start/finish region failed to close into a clean loop. Driving slowly lets the scan matcher converge between keyframes and yields a closed-loop, drift-free map on a single lap.

[1]Original repository: https://github.com/mit-racecar/particle_filter; we forked at the master branch and built against ROS 2 Humble.

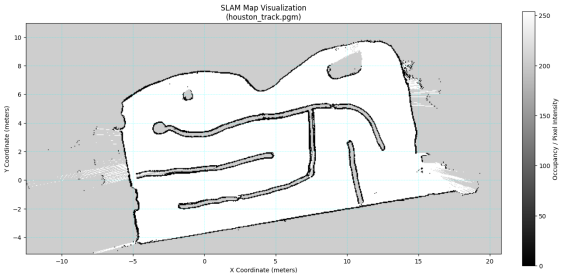

Fig. 2: Final occupancy grid map produced by `slam_toolbox` in `online_async` mode after a single 1 m/s teleoperated lap.

Foxglove Studio is used purely as a visualization client during mapping — to watch the live map, the LiDAR scan, and the TF tree — and is not in the data recording path. All bag and map artifacts are produced by `slam_toolbox` itself.

Map post-processing is intentionally minimal. Rather than scrubbing the map by hand, we treat the map as acceptable if (i) the track loops visibly close in RViz, and (ii) localization with the particle filter described below converges on the first attempt and the vehicle can be driven around the full lap under human teleoperation while remaining on its expected pose. If both checks pass the map is frozen and used for the rest of the project.

IV. Localization

Online localization is provided by a fork of the MIT-RACECAR `particle_filter`, a Monte Carlo localization implementation purpose-built for F1TENTH-class vehicles. We deliberately avoid `nav2_amcl` because the MIT filter is tuned for high-rate, narrow-FoV planar LiDAR and exposes a ray-marched range likelihood that is well matched to the corridor-like geometry of an indoor track.

In our configuration, the filter sustains an effective particle update rate of **≈40 Hz** on the Jetson Orin Nano Super with the default particle count, and publishes its pose estimate on `/pf/viz/inferred_pose` as a `geometry_msgs/PoseStamped`. The pure pursuit node consumes either this topic or simulated odometry on `/ego_racecar/odom`, controlled by the boolean `use_odom` parameter. We did not modify the filter's measurement model or motion model beyond the upstream defaults; tuning was concentrated downstream in the controller.

V. Waypoint Generation and Curation

A. Raw waypoint collection

Once the map is frozen and the particle filter is running, a second teleoperated lap is driven at low speed and the inferred pose is logged at every update. This produces a dense, slightly noisy raw raceline of several hundred poses tracing roughly the centerline of the track.

B. Manual curation strategy

Rather than fitting a parametric racing line, we down-sample and reshape the raw trace by hand using the editor described in Sec. V-D. Two qualitative principles guide curation:

1) *Density follows curvature.* Corners receive a denser concentration of waypoints, with a waypoint at corner entry, apex, and exit, while straights are represented by a small number of widely spaced anchors. The result is that the heading rate between adjacent waypoints stays roughly bounded.
2) *The line is shaped, not fit.* Because all subsequent obstacle avoidance happens through lane selection rather than through online replanning, the racing line itself can afford to be aggressive. Apexes are cut tightly, exits are pushed to the outside of the track, and the entry of each corner is biased late.

The final curated raceline contains approximately **30 waypoints per lane**.

C. Velocity profiling

Each waypoint carries a hand-assigned target speed in its fourth column. The profile was tuned iteratively against the simulator and the physical track. Two heuristics produced the largest improvements:

- Straights are run at up to **7 m/s**, the maximum at which the controller maintains stable tracking under our gain and lookahead settings.
- Corners are entered with a single *aggressive deceleration* waypoint placed just before the apex with a target speed of **2 m/s**, even when the rest of the corner runs at ≈ 3 m/s. We found that placing one slow waypoint at corner entry let the vehicle scrub speed sharply, settle, and then carry meaningful momentum through the apex; without it, the controller would either understeer wide, or, at higher commanded speeds, overshoot the apex and clip the inside wall.

This per-waypoint profile is consumed by the controller in two ways: (i) the target speed is published directly to `/drive` after multiplication by the opponent-aware speed scale (Sec. VI-G), and (ii) it feeds the adaptive lookahead computation when the controller is not driven from odometry-derived speed.

D. Interactive waypoint editor

The editor is a single-file Python application built on Matplotlib's event callbacks. The map produced in Sec. III is displayed as a grayscale background; waypoints are rendered as draggable markers colored by lane index, with each marker labeled by its target speed. Mouse interactions support inserting a new waypoint between two existing ones, deleting a waypoint, dragging a waypoint to a new position, and editing the speed value through a keyboard shortcut. On save, the editor exports a CSV with columns `x, y, v, lane_id` that the controller loads at startup.

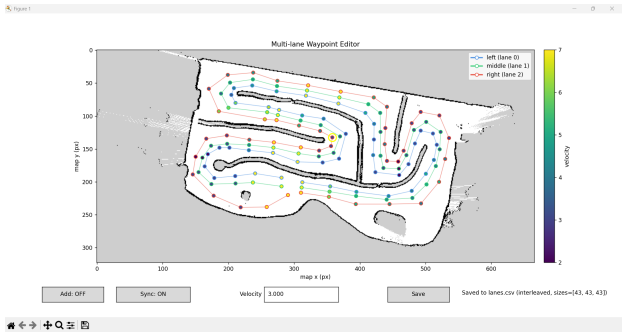

Fig. 3: Interactive waypoint editor. The SLAM occupancy grid is drawn as the background; markers can be dragged, inserted, deleted, and assigned per-point target speeds before being exported as a CSV.

E. Multi-lane construction

The controller is written against a fixed three-lane interface (lane 0 is the leftmost candidate, lane 1 the middle, lane 2 the rightmost). During the project we experimented with both a two-raceline configuration and a full three-raceline configuration. The final competition setup used two physical racelines plus a duplicated copy to satisfy the controller's three-lane contract; lane indexing is preserved so that for any index i, the entries `lanes[0][i]`, `lanes[1][i]`, `lanes[2][i]` are aligned in arc-length and form a valid switch triple. The CSV format `x, y, v, lane_id` encodes lane membership per-row so that all racelines live in a single file.

VI. Multi-Lane Pure Pursuit Controller

This section describes the runtime controller in detail. The implementation is a single ROS 2 node, `pure_pursuit_node`, written in C++ against `rclcpp`. It subscribes to a pose topic (either simulator odometry or particle-filter pose) and a LaserScan topic, and publishes `AckermannDriveStamped` drive commands together with three visualization topics for waypoints, the active goal, and detected opponents.

A. Pure pursuit fundamentals

Let the vehicle pose in the map frame be (p_x, p_y, ψ), where the heading ψ is recovered from the orientation quaternion (q_x, q_y, q_z, q_w) via the standard ZYX yaw extraction

$$\psi = \operatorname{atan2}\big(2(q_w q_z + q_x q_y),\ 1 - 2(q_y^2 + q_z^2)\big)\,. \tag{1}$$

Given a goal point (g_x, g_y) in the map frame, its coordinates in the vehicle's body frame are

$$g_x^v = \ \ (g_x - p_x)\cos\psi + (g_y - p_y)\sin\psi, \tag{2}$$
$$g_y^v = -(g_x - p_x)\sin\psi + (g_y - p_y)\cos\psi. \tag{3}$$

With effective lookahead distance $L_{\text{act}} = \sqrt{(g_x^v)^2 + (g_y^v)^2}$, the path curvature required to intersect the goal point is

$$\kappa = \frac{2\, g_y^v}{L_{\text{act}}^2}, \tag{4}$$

and the bicycle steering command, with wheelbase C_L, is

$$\delta = \mathrm{clamp}(\arctan(\kappa\, C_L),\ -\delta_{\max},\ \delta_{\max})\,. \tag{5}$$

This is the classical Coulter formulation [1].

B. Adaptive lookahead

A fixed lookahead distance trades off cornering precision against straight-line stability poorly: short lookaheads oscillate at high speed, long lookaheads cut corners. We instead compute a lookahead that scales linearly with a reference speed v_{ref}:

$$L = \mathrm{clamp}(k_L\, v_{\mathrm{ref}},\ L_{\min},\ L_{\max})\,, \tag{6}$$

where k_L is the `lookahead_gain` parameter. The reference speed is defined to fall back gracefully when the vehicle is stationary or when the upstream speed estimate is unavailable:

$$v_{\mathrm{ref}} = \begin{cases} \max(v_{\mathrm{cur}},\ v_{\mathrm{floor}}) & \text{if odometry,} \\ \max(\max(v_{\mathrm{wp}}, v_{\mathrm{last}}),\ v_{\mathrm{floor}}) & \text{otherwise,} \end{cases} \tag{7}$$

where v_{cur} is the current measured speed, v_{wp} is the curated speed at the nearest waypoint on the active lane, v_{last} is the most recent published command speed, and v_{floor} is the `min_speed_for_lookahead` parameter.

C. Goal selection on the active lane

Given the active lane ℓ and the index $i^\star$ of the waypoint nearest to the current pose, the goal point is the first waypoint encountered when walking forward along the lane whose Euclidean distance from the current pose exceeds the adaptive lookahead L:

$$i_{\mathrm{goal}} = \min\{k \geq i^\star : \|w_{\ell,k} - p\| \geq L\}\,, \tag{8}$$

where $w_{\ell,k} = (x, y)$ denotes the k-th waypoint of lane ℓ and indices wrap around the lap. The goal is then transformed by (2)–(3) and passed to (4)–(5).

D. LiDAR-based opponent detection

Opponent detection runs on every `LaserScan` callback. The procedure has three stages: range-jump segmentation, geometric cluster validation in the vehicle frame, and a transformation into the map frame.

a) Segmentation.: Consecutive scan returns are grouped into a cluster as long as the absolute change between adjacent ranges is small,

$$|r_i - r_{i-1}| < \Delta R_{\mathrm{jump}}, \tag{9}$$

where $\Delta R_{\mathrm{jump}} = 0.20\,\mathrm{m}$. A cluster is also flushed whenever a range is non-finite, below the LiDAR's minimum range, or above the cutoff $R_{\max} = 6.0\,\mathrm{m}$.

b) Cluster validation.: A cluster is accepted only if it satisfies:

- a point-count check: $N_{\min} \leq |\mathcal{C}| \leq N_{\max}$ with $N_{\min} = 3$, $N_{\max} = 40$;
- a chord-width check on the cluster endpoints,

$$w_{\mathrm{cluster}} = \sqrt{(x_1 - x_0)^2 + (y_1 - y_0)^2} \in [w_{\min},\, w_{\max}], \tag{10}$$

with $w_{\min} = 0.03\,\mathrm{m}$ and $w_{\max} = 0.40\,\mathrm{m}$. The upper bound corresponds to an F1TENTH-class chassis footprint of $\approx 12\,\mathrm{cm}$ with generous tolerance for clustering noise.

Walls and long flat surfaces fail (10) by being too wide; sparse single-beam returns from spurious reflections fail the point-count test.

c) Frame transformation.: The cluster centroid in the vehicle frame is

$$\big(c_x^v, c_y^v\big) = \frac{1}{N_{\mathrm{valid}}} \sum_{k \in \mathcal{C}} \big(r_k \cos\theta_k,\ r_k \sin\theta_k\big), \tag{11}$$

where $\theta_k = \theta_{\min} + k\,\Delta\theta$. It is then mapped into the global frame using the cached pose:

$$m_x = p_x + c_x^v \cos\psi - c_y^v \sin\psi, \tag{12}$$

$$m_y = p_y + c_x^v \sin\psi + c_y^v \cos\psi. \tag{13}$$

E. Cross-frame persistence

Single-frame detections are noisy: a momentary occlusion or a glancing reflection produces a single-frame phantom that, if acted on directly, would trigger spurious lane switches. We confirm a detection only if a similar detection was present in the immediately previous LiDAR frame, where "similar" means a Euclidean distance under $r_{\mathrm{assoc}} = 0.5\,\mathrm{m}$ in the map frame:

$$\mathcal{O}_{\mathrm{conf}} = \big\{d \in \mathcal{D}_t :\ \exists\, d' \in \mathcal{D}_{t-1},\ \|d - d'\| < r_{\mathrm{assoc}}\big\}, \tag{14}$$

where $\mathcal{D}_t$ are the raw detections at frame t. The current implementation uses two-frame persistence; a longer window is straightforward but costs detection latency.

Algorithm 1 summarizes the full opponent detection pipeline.

F. Lane scoring and selection

Lane selection is recomputed on every pose update. Given the index $i^\star$ of the waypoint nearest to the current pose on the currently active lane, we form, for each candidate lane $\ell \in \{0, 1, 2\}$, a forward-window minimum clearance against the confirmed opponent set:

$$c(\ell) = \min_{\substack{k:\, s_k \leq H \\ o \in \mathcal{O}_{\mathrm{conf}}}} \big\|w_{\ell, i^\star + k} - o\big\|, \tag{15}$$

where $s_k = \sum_{j=0}^{k-1} \|w_{\ell,i^\star+j+1} - w_{\ell,i^\star+j}\|$ is the accumulated arc length and $H = 3.0\,\mathrm{m}$ is the `scoring_horizon`. If $\mathcal{O}_{\mathrm{conf}} = \emptyset$ then $c(\ell) = +\infty$. From this clearance we derive a smooth collision cost,

$$J_{\mathrm{coll}}(\ell) = \begin{cases} W_{\mathrm{coll}}, & c(\ell) < c_{\mathrm{safe}}, \\ \exp(-2(c(\ell) - c_{\mathrm{safe}}))\,, & c(\ell) \geq c_{\mathrm{safe}}, \end{cases} \tag{16}$$

Algorithm 1 LiDAR opponent detection (per scan)

Require: LaserScan $r_{0..n-1}$, pose (p_x, p_y, ψ), previous detections $\mathcal{D}_{t-1}$

$\mathcal{C} \leftarrow \emptyset,\ \mathcal{D}_t \leftarrow \emptyset$
for $i = 0$ to $n-1$ **do**
  if r_i not finite **or** $r_i \le r_{\min}$ **or** $r_i > R_{\max}$ **then**
    FLUSH($\mathcal{C}$); **continue**
  end if
  if $\mathcal{C} = \emptyset$ **or** $|r_i - r_{\mathcal{C}.\text{last}}| < \Delta R_{\text{jump}}$ **then**
    $\mathcal{C} \leftarrow \mathcal{C} \cup \{i\}$
  else
    FLUSH($\mathcal{C}$); $\mathcal{C} \leftarrow \{i\}$
  end if
end for
FLUSH($\mathcal{C}$)
$\mathcal{O}_{\text{conf}} \leftarrow \emptyset$
for all $d \in \mathcal{D}_t$ **do**
  if $\exists\, d' \in \mathcal{D}_{t-1} : \|d - d'\| < r_{\text{assoc}}$ **then**
    $\mathcal{O}_{\text{conf}} \leftarrow \mathcal{O}_{\text{conf}} \cup \{d\}$
  end if
end for
$\mathcal{D}_{t-1} \leftarrow \mathcal{D}_t$; **return** $\mathcal{O}_{\text{conf}}$

function FLUSH($\mathcal{C}$)
  if $|\mathcal{C}| < N_{\min}$ **or** $|\mathcal{C}| > N_{\max}$ **then**
    $\mathcal{C} \leftarrow \emptyset$; **return**
  end if
  compute endpoint chord width w_{cluster} via (10)
  if $w_{\text{cluster}} \notin [w_{\min}, w_{\max}]$ **then**
    $\mathcal{C} \leftarrow \emptyset$; **return**
  end if
  compute centroid (c_x^v, c_y^v) via (11)
  map to global frame via (12)–(13) $\rightarrow (m_x, m_y)$
  $\mathcal{D}_t \leftarrow \mathcal{D}_t \cup \{(m_x, m_y)\}$; $\mathcal{C} \leftarrow \emptyset$
end function

with hard penalty $W_{\text{coll}} = 100.0$ and safety threshold $c_{\text{safe}} = 0.35\,\text{m}$. The exponential decay is what makes the selector *prefer* clear lanes in proportion to their clearance, rather than treating any $c(\ell) \ge c_{\text{safe}}$ as identically safe.

The total per-lane cost adds two further regularizers:

$$J(\ell) = J_{\text{coll}}(\ell) + W_{\text{LR}}\,\ell + W_{\text{sw}}\,\mathbf{1}[\ell \neq \ell_{\text{act}}], \tag{17}$$

with $W_{\text{LR}} = 0.5$ and $W_{\text{sw}} = 0.3$. The first regularizer is a leftward racing-line preference: with our lane indexing, lower indices are preferred all else equal, and this matches the dominant turn direction of the competition track. The second is a switching penalty that makes the controller stick to the currently active lane when the costs of two lanes are similar.

A cooldown timer of $T_{\text{cool}} = 0.5\,\text{s}$ suppresses high-frequency lane oscillation, but is overridden when the active lane is itself blocked ($c(\ell_{\text{act}}) < c_{\text{safe}}$) so that emergency switches are never delayed. Algorithm 2 states the selection rule precisely.

Algorithm 2 Lane selection at each pose update

Require: nearest index $i^\star$, opponent set $\mathcal{O}_{\text{conf}}$, active lane ℓ_{act}, last switch time t_{sw}

for $\ell \in \{0, 1, 2\}$ **do**
  compute $c(\ell)$ via (15)
  $\text{blocked}[\ell] \leftarrow c(\ell) < c_{\text{safe}}$
  compute $J_{\text{coll}}(\ell)$ via (16)
  $J(\ell) \leftarrow J_{\text{coll}}(\ell) + W_{\text{LR}}\ell + W_{\text{sw}}\mathbf{1}[\ell \neq \ell_{\text{act}}]$
end for
$\ell^\star \leftarrow \arg\min_\ell J(\ell)$
$\Delta t \leftarrow t - t_{\text{sw}}$
if $\ell^\star \neq \ell_{\text{act}}$ **and** ($\Delta t \ge T_{\text{cool}}$ **or** $\text{blocked}[\ell_{\text{act}}]$) **then**
  $\ell_{\text{act}} \leftarrow \ell^\star;\ t_{\text{sw}} \leftarrow t$
end if
return ℓ_{act}

G. Speed modulation in the presence of an opponent

Selecting a clear lane is necessary but not sufficient: even on the active lane, a slower opponent ahead must be matched in speed before it can be passed. We therefore scale the published target speed by a factor that depends on the distance to the closest opponent that lies on or near the active lane within a following window. Concretely, we walk forward along the active lane, accumulating arc length s, and record the smallest s at which any opponent lies within $1.5\,c_{\text{safe}}$ of a waypoint. Calling this distance d_{ahead} and the parameter $D_{\text{follow}} = 1.5\,\text{m}$, the speed scale is

$$s_{\text{scale}} = s_{\min} + (1 - s_{\min})\,\text{clamp}\left(\frac{d_{\text{ahead}}}{D_{\text{follow}}}, 0, 1\right), \tag{18}$$

with $s_{\min} = 0.3$. When no opponent is ahead, $d_{\text{ahead}} = +\infty$ and $s_{\text{scale}} = 1$. The published speed command is then

$$v_{\text{cmd}} = v_{\text{wp}}(g_{\text{idx}}) \cdot s_{\text{scale}}. \tag{19}$$

H. Putting it together

Algorithm 3 shows the per-pose-update control step. Every callback re-localizes on the active lane, calls the lane selector, re-localizes on the (possibly new) active lane, computes the adaptive lookahead, picks a goal, solves (2)–(5) for the steering command, and computes the speed command via (18)–(19). The corresponding `AckermannDriveStamped` message is then published.

- `/waypoints_viz`: all waypoints across all three lanes, color coded (lane 0 cyan, lane 1 yellow, lane 2 magenta), with the active lane drawn at full opacity and larger radius;
- `/goal_waypoint_viz`: the currently selected goal point as a red sphere;
- `/opponents_viz`: confirmed opponents as orange cubes published with `DELETEALL` cleanup so that stale cubes never accumulate.

These three streams together let an operator diagnose, in real time, whether a crash is due to localization, lane selection, opponent detection, or pure pursuit gain mistuning.

Algorithm 3 Per-pose-update control step

Require: pose (p_x, p_y, ψ), current speed v_{cur}, opponents $\mathcal{O}_{\text{conf}}$, lanes $\{\mathcal{L}_0, \mathcal{L}_1, \mathcal{L}_2\}$

$i^\star \leftarrow \arg\min_i \|w_{\ell_{\text{act}},i} - (p_x, p_y)\|$
$\ell_{\text{act}} \leftarrow$ SELECTLANE($i^\star$, $\mathcal{O}_{\text{conf}}$, ℓ_{act}) ▷ Algorithm 2
$i^\star \leftarrow \arg\min_i \|w_{\ell_{\text{act}},i} - (p_x, p_y)\|$
$L \leftarrow$ adaptive lookahead via (6)–(7)
$i_{\text{goal}} \leftarrow$ first $k \geq i^\star$ with $\|w_{\ell_{\text{act}},k} - p\| \geq L$
$(g_x^v, g_y^v) \leftarrow$ transform via (2)–(3)
$L_{\text{act}} \leftarrow \sqrt{(g_x^v)^2 + (g_y^v)^2}$; **if** $L_{\text{act}} < 10^{-3}$ **then return**
$\kappa \leftarrow 2g_y^v / L_{\text{act}}^2$
$\delta \leftarrow \text{clamp}(\arctan(\kappa C_L), -\delta_{\max}, \delta_{\max})$
$s_{\text{scale}} \leftarrow$ via (18)
$v_{\text{cmd}} \leftarrow v_{\text{wp}}(i_{\text{goal}}) \cdot s_{\text{scale}}$
publish `AckermannDriveStamped`(δ, v_{cmd})

TABLE II: Pure Pursuit Node Parameters (ROS 2)

Parameter	Value	Meaning
`lookahead_distance`	0.8 m	fixed-mode lookahead (off)
`lookahead_gain` (k_L)	0.5 s	adaptive lookahead slope
`min_lookahead`	0.5 m	adaptive lower bound
`max_lookahead`	2.0 m	adaptive upper bound
`velocity`	0.8 m/s	fallback speed
`max_steering_angle`	0.4189 rad	$\delta_{\max}$
`wheelbase` (C_L)	0.3302 m	vehicle wheelbase
`use_odom`	true / false	sim vs. hardware
`speed_lookahead`	true	enable adaptive lookahead
`min_speed_for_lookahead`	0.5 m/s	v_{floor}
`scan_topic`	`/scan`	LiDAR topic
`scoring_horizon_m` (H)	3.0 m	lane lookahead window
`safety_threshold_m` (c_{safe})	0.35 m	blocking radius
`follow_distance_m` (D_{follow})	1.5 m	speed-scale ramp
`min_speed_scale` ($s_{\min}$)	0.3	lower speed scale

VII. System Architecture and Implementation

The runtime graph contains four logical actors: the driver layer (VESC and LiDAR), the localizer (`particle_filter` on hardware, `ego_racecar/odom` in simulation), the controller (`pure_pursuit_node`), and the simulator or vehicle bring-up.

The full controller parameter set is listed in Table II; the hardcoded compile-time constants for opponent detection and lane scoring, which we deliberately did not expose as parameters in order to lock down behavior between testing sessions, are listed in Table III.

VIII. Experimental Results

The controller was first tuned in the F1TENTH ROS 2 gym and subsequently deployed on the physical car. Testing was structured around *time trials* on the empty track — with the goal of minimizing single-lap completion time along our own raceline — and a small number of *head-to-head* runs against a competing agent during the in-class race.

TABLE III: Hardcoded Detection & Scoring Constants

Constant	Value	Role
`SWITCH_COOLDOWN_S`	0.5 s	min interval between switches
`CLUSTER_RANGE_JUMP_M`	0.20 m	seg. threshold (9)
`CLUSTER_MIN_POINTS`	3	cluster $N_{\min}$
`CLUSTER_MAX_POINTS`	40	cluster $N_{\max}$
`CLUSTER_MAX_WIDTH_M`	0.40 m	$w_{\max}$
`CLUSTER_MIN_WIDTH_M`	0.03 m	$w_{\min}$
`CLUSTER_MAX_RANGE_M`	6.0 m	$R_{\max}$
`LEFTWARD_PREFERENCE_W`	0.5	W_{LR}
`SWITCHING_PENALTY_W`	0.3	W_{sw}
`COLLISION_PENALTY_W`	100.0	W_{coll}
`OPP_PERSISTENCE_FRAMES`	2	two-frame persistence
`OPP_ASSOC_RADIUS_M`	0.5 m	r_{assoc}

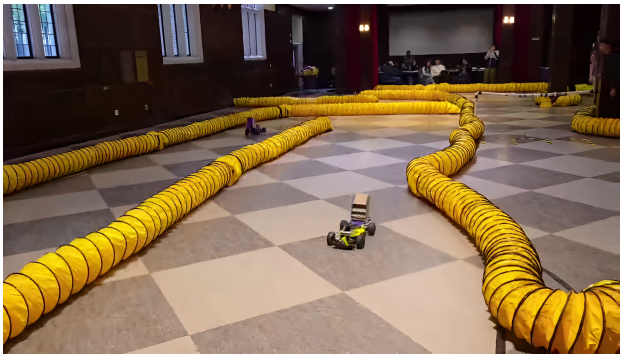

Fig. 4: Recorded onboard footage of the best-time race lap. (Click the image to view the video)

A. Tuning trajectory

Early laps were dominated by understeer-induced collisions at the same two corners. The root cause was always the same: the curated waypoint speed at corner entry was too high, the front tires saturated, and the vehicle drifted into the outside wall. The fix was the corner-entry deceleration heuristic described in Sec. V: dropping the first waypoint of each significant corner to 2 m/s reliably eliminated the understeer. Once that single change landed, the vehicle could complete laps without crashing in both the simulator and on hardware. From that point onward, all further tuning was concerned with shaving lap time rather than fixing failure modes: raising straight-line targets toward 7 m/s, tightening apex placement, and nudging the corner-exit waypoints slightly outside.

B. Best lap time

The best lap recorded during the in-class head-to-head race was **21 s**. This was a competitive run, not a clean time trial; the vehicle nevertheless achieved its best lap on the track over all the days of testing during the race itself.

C. Notes on baseline comparison

We do *not* report a single-lane baseline. The multi-lane controller was treated as the design baseline from the beginning of the project; ablation of the lane-switching machinery was never run in a controlled time-trial setting, and reporting an

unverified ablation would be misleading. We acknowledge this as a limitation of the experimental section and revisit it in Sec. X.

IX. Limitations and Failure Analysis

Three classes of failure are worth recording explicitly.

A. Spurious lane oscillation in the absence of opponents

During time trials, with no opponent on the track, the active lane would occasionally toggle between candidates — driven not by detected obstacles (of which there were none) but by small clearance differences in (16) interacting with the leftward preference and the switching penalty. The cooldown timer suppresses the worst of this, but does not eliminate it. The functional consequence is that the vehicle does not always follow our nominally-optimal raceline even when it should; instead, it drifts between candidate racelines and pays a small but nonzero lap-time penalty. We believe a cleaner fix is to gate the lane scorer on the existence of at least one confirmed opponent and pin the active lane to a designated "primary" raceline otherwise. We did not have the testing budget to validate this change before the race.

B. Side-on collision blind spot

In one head-to-head run, the ego vehicle was contacted on the side, near the rear, by an opposing agent during the first corner. A 2D LiDAR mounted at the front of the vehicle has limited visibility past the chassis itself toward the rear and rear-quarter; a side-on, slightly trailing approach is therefore effectively invisible to opponent detection until the moment of contact. This is a sensor-coverage limitation, not a controller bug, and is shared by every F1TENTH stack that relies solely on a planar front LiDAR. It is most cleanly addressed by adding a rear-facing range sensor or a second LiDAR.

C. Limited testing duration

Practice time on the physical track was short. We were able to converge on a working velocity profile, but did not have the testing budget to run a proper raceline optimization on top of the manually-shaped lines, nor to do a controlled ablation of the lane scorer's individual cost terms. Most of the parameter values in Tables II–III are the result of a single-pass tuning effort, not a sweep; they are likely good but unlikely to be optimal.

X. Future Work

Three directions are concrete enough to call out.

Gap-following on opponent detection. A simple and likely effective upgrade is to keep the multi-lane pure pursuit running until an opponent is confirmed, and only then hand off to a follow-the-gap controller for the duration of the engagement. This decouples the two regimes that the current controller blurs: pure raceline tracking and reactive avoidance.

Learning-based opponent prediction with a properly-tuned tracker. The two-frame persistence filter is the absolute minimum sufficient to suppress single-frame noise. Replacing it with a small Kalman filter or constant-turn-rate motion model on confirmed clusters would yield smoothed opponent positions and, more importantly, opponent velocities. With predicted opponent trajectories we could score lanes on the basis of *future* clearance rather than instantaneous clearance, which directly addresses the oscillation pathology described in Sec. IX: a lane that is about to be occupied should be costed similarly to a lane currently occupied, not the same as a lane that will be clear.

Raceline optimization. The hand-curated racelines are aggressive but not provably optimal. A standard offline minimum-curvature or minimum-time raceline optimizer running on the post-SLAM track polygon would likely yield several tenths per lap relative to the manual lines, particularly on the long double-apex turn where our lines were pieced together by visual inspection.

XI. Conclusion

We have presented a multi-lane pure pursuit controller for F1TENTH-class autonomous racing. The system is intentionally simple — a fixed set of hand-curated racelines, a classical pure pursuit law with adaptive lookahead, and a per-frame cost minimization over lane indices — but the integration of LiDAR clustering with cross-frame persistence, a smooth collision-clearance cost with a hard safety threshold, a switching penalty under a cooldown, and an opponent-aware speed scale gives a controller that is robust enough to race head-to-head on real hardware. The vehicle achieved a 21 s best lap during the competition. The most useful single design lesson, for those attempting to reproduce this system, is the corner-entry deceleration heuristic: a single low-speed waypoint placed just before the apex outweighs substantial controller-side gain tuning.

Acknowledgments

The author thanks the F1TENTH course staff at the University of Pennsylvania for providing the vehicle, simulator, and track time, and for their support throughout the project.

References

[1] R. C. Coulter, "Implementation of the Pure Pursuit Path Tracking Algorithm," Carnegie Mellon University, The Robotics Institute, Pittsburgh, PA, USA, Tech. Rep. CMU-RI-TR-92-01, January 1992.